UFRN – PROFª SÍLVIA MAIA

EXERCÍCIOS - COMPLEXIDADE

1) Escreva as seguintes funções em notação *O*:
   a) $n^3 - 1$; b) $n^2 + 2 \log n$; c) $3n^n + 5 \cdot 2^n$ d) $(n\text{-}1)^n + n^{n-1}$

2) Considerando o polinômio quadrático $q(n) = 5n^2 + 7n + 3$, mostre que $q(n) = O(n^2)$, determinando constantes $n_0 \in \aleph$ e $c \in \Re_+$ tais que $q(n) \leq cn^2$, para todo $n \geq n_0$.

3) Qual(ais) das seguintes afirmações sobre o crescimento assintótico de funções não é(são) verdadeira(s)? Justifique.
   a) $2n^2 + 3n + 1 = O(n^2)$
   b) $n^2 = \Omega(n^3)$
   c) Se $f(n) = O(g(n))$ então $g(n) = O(f(n))$
   d) $\log n^2 = O(\log n)$
   e) Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$
   f) $2^{n+1} = O(2^n)$
   g) $n! = \Theta((n + 1)!)$

4) Projete um algoritmo para determinar o máximo e o mínimo de um vetor vet de valores e analise suas complexidades de pior caso.

5) Considere que, inicialmente, são dadas n listas ordenadas, cada uma com tamanho m_i, $1 \leq i \leq n$. Desenvolva um algoritmo que una as listas dadas de maneira que a única lista resultante também esteja ordenada. O algoritmo deve ser o mais eficiente possível. Determine a complexidade de pior caso para o algoritmo.

6) Considere dois algoritmos *A* e *B* que apresentam tempo em $\Theta(n^2)$ e $\Theta(n^3)$, respectivamente, para solucionar um dado problema. Se recursos como memória e tempo de programação não são considerados, é necessariamente verdade que o algoritmo *A sempre* é preferível ao algoritmo *B*? Justifique sua resposta.

7) Analise a complexidade local dos algoritmos a seguir e a expresse na notação assintótica Θ.

```
l ← 0
para  i ← 1 até n faça
    para j ← 1 até n² faça
        para k ← 1 até n³ faça
            l ← l + 1
```

```
l ← 0
para  i ← 1 até n faça
    para j ← 1 até i faça
        para k ← j até n faça
            l ← l + 1
```

8) Encontre funções $f_1(n)$ e $f_2(n)$ tais que sejam $O(g(n))$, mas $f_1(n)$ não seja $O(f_2(n))$.

9) Mostre que, se c é um número real positivo, então $g(n) = 1 + c + c^2 + ... + c^n$ é:
   a) $\Theta(1)$ se $c < 1$
   b) $\Theta(n)$ se $c = 1$
   c) $\Theta(c^n)$ se $c > 1$

10) Os algoritmos A_1 e A_2 abaixo resolvem o mesmo problema. Eles recebem como entrada os valores de n (inteiro) e x (real) e um vetor a de tamanho n+1 contendo valores reais. Os algoritmos então calculam o valor do polinômio $p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0$.
   a) Faça a análise de complexidade local e assintótica dos dois algoritmos.
   b) Qual deles é assintoticamente mais eficiente?

```c
void a1 (int n, float x, float a[]) {
    float p;
    int i;
    p = a[0];
    for (i=1; i<=n; i++)
        p = p + a[i]*pow(x,i);
    printf ("%.2f\n", p);
}
```

```c
void a2 (int n, float x, float a[]) {
    float p, y;
    int i;
    p = a[0];
    y = x;
    for (i=1; i<n; i++) {
        p = p + a[i]*y;
        y = y * x;
    }
    p = p + a[n]*y;
    printf ("%.2f\n", p);
}
```

11) Sejam $T_1(n) = \sqrt{n}\ (log_2 n)$; $T_2(n) = n$. Responda às seguintes questões, justificando:
   i) $T_1(n)$ é $O(T_2(n))$?
   ii) $T_1(n)$ é $\Omega(T_2(n))$?
   iii) $T_1(n)$ é $\Theta(T_2(n))$?
   iv) $T_1(n)$ é $o(T_2(n))$?
   v) $T_1(n)$ é $\omega(T_2(n))$?